

▼ Application_Logs

Data Loading

```
pip install pandas

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("Application_Logs_175000_Rows.csv")
```

```
print(df.head())
```

	Log_ID	Timestamp	Application_Name	Environment	Server_Name	\
0	1	2026-01-01 00:00:00	CRM	QA	APP-SRV-02	
1	2	2026-01-01 00:01:00	Inventory	Production	APP-SRV-01	
2	3	2026-01-01 00:02:00	ECommerce	QA	APP-SRV-03	
3	4	2026-01-01 00:03:00	HRMS	Production	APP-SRV-04	
4	5	2026-01-01 00:04:00	HRMS	Production	APP-SRV-02	

	Host_IP	User_ID	Session_ID	Log_Level	Module	Event_Type	\
0	192.168.215.248	40754	SES189608	DEBUG	Database	Insert	
1	192.168.216.216	13718	SES703197	ERROR	Database	API_Call	
2	192.168.213.171	16536	SES942004	ERROR	Payment	Insert	
3	192.168.21.62	20758	SES514486	CRITICAL	Payment	Logout	
4	192.168.228.124	33203	SES342643	CRITICAL	Database	Delete	

	Response_Time_ms	Status_Code	CPU_Usage_Percent	Memory_Usage_MB	\
0	799	201	91.77	5248	
1	2634	403	20.27	13792	
2	1815	401	49.43	5444	
3	1324	404	8.39	15420	
4	4032	403	53.23	10779	

	Disk_Usage_Percent	Request_Size_KB	Response_Size_KB	Error_Code	\
0	53.14	3693.02	7790.05	ERR404	
1	91.92	2307.82	6317.57	ERR303	
2	98.90	3882.70	8514.84	ERR101	
3	45.53	1954.54	2966.26	NaN	
4	75.94	2692.02	9897.03	NaN	

	Message
0	Invalid credentials
1	Transaction completed
2	Timeout occurred
3	Database connection established
4	Resource not found

```
print(df.tail())
```

	Log_ID	Timestamp	Application_Name	Environment	Server_Name	\
174995	174996	2026-05-02 12:35:00	CRM	QA	APP-SRV-04	
174996	174997	2026-05-02 12:36:00	Inventory	UAT	APP-SRV-03	
174997	174998	2026-05-02 12:37:00	Inventory	Development	APP-SRV-04	
174998	174999	2026-05-02 12:38:00	Banking	Production	APP-SRV-04	
174999	175000	2026-05-02 12:39:00	ECommerce	Production	APP-SRV-01	

	Host_IP	User_ID	Session_ID	Log_Level	Module	\
174995	192.168.124.158	16993	SES993272	ERROR	Authentication	
174996	192.168.4.49	9801	SES534097	INFO	Authentication	
174997	192.168.119.91	15443	SES576538	INFO	Orders	

174998	192.168.60.113	35713	SES896132	DEBUG	Reporting
174999	192.168.34.160	2315	SES268988	INFO	Orders
	Event_Type	Response_Time_ms	Status_Code	CPU_Usage_Percent	
174995	Logout	1099	400	27.99	
174996	Login	309	400	20.12	
174997	Insert	929	400	93.76	
174998	Insert	1739	200	33.88	
174999	Logout	3390	200	54.84	
	Memory_Usage_MB	Disk_Usage_Percent	Request_Size_KB	\	
174995	7702	29.43	2153.84		
174996	7044	13.53	4298.52		
174997	1259	94.85	4990.47		
174998	13173	12.25	1235.47		
174999	14405	32.73	2412.93		
	Response_Size_KB	Error_Code	Message		
174995	3714.35	ERR202	Timeout occurred		
174996	8805.31	ERR404	Transaction completed		
174997	1046.74	ERR101	Database	connection established	
174998	3083.86	ERR101	API request received		
174999	6238.52	ERR202	Invalid credentials		

```
print(df.shape)
```

```
(175000, 20)
```

```
print(df.columns)
```

```
Index(['Log_ID', 'Timestamp', 'Application_Name', 'Environment', 'Server_Name',  
      'Host_IP', 'User_ID', 'Session_ID', 'Log_Level', 'Module', 'Event_Type',  
      'Response_Time_ms', 'Status_Code', 'CPU_Usage_Percent',  
      'Memory_Usage_MB', 'Disk_Usage_Percent', 'Request_Size_KB',  
      'Response_Size_KB', 'Error_Code', 'Message'],  
      dtype='object')
```

```
print(df.dtypes)
```

```
Log_ID          int64  
Timestamp      object  
Application_Name object  
Environment     object  
Server_Name     object  
Host_IP        object  
User_ID        int64  
Session_ID     object  
Log_Level      object  
Module         object  
Event_Type     object  
Response_Time_ms int64  
Status_Code    int64  
CPU_Usage_Percent float64  
Memory_Usage_MB int64  
Disk_Usage_Percent float64  
Request_Size_KB float64  
Response_Size_KB float64  
Error_Code     object  
Message        object  
dtype: object
```

```
df['Timestamp'] = pd.to_datetime(df['Timestamp'])  
df.set_index('Timestamp', inplace=True)
```

Data Quality Checks

```
print(df.isnull().sum())
```

```
Log_ID          0  
Application_Name 0  
Environment     0  
Server_Name     0  
Host_IP        0  
User_ID        0  
Session_ID     0  
Log_Level      0  
Module         0  
Event_Type     0  
Response_Time_ms 0
```

```
Status_Code          0
CPU_Usage_Percent    0
Memory_Usage_MB      0
Disk_Usage_Percent   0
Request_Size_KB      0
Response_Size_KB     0
Error_Code           29007
Message              0
dtype: int64
```

```
print(df.duplicated().sum())
```

```
0
```

```
valid_codes=[200,201,400,401,403,404,500,503]
invalid_records = df[
    ~df['Status_Code'].isin(valid_codes)
]
print(len(invalid_records))
```

```
0
```

```
print(df['Session_ID'].isnull().sum())
```

```
0
```

```
invalid_users = df[df['User_ID']<= 0]
print(len(invalid_users))
```

```
0
```

```
future_logs = df[
    df.index > pd.Timestamp.now()
]
print(len(future_logs))
```

```
0
```

```
memory = df.memory_usage(
    deep = True
).sum()

print(memory/1024/1024)
```

```
109.65381717681885
```

```
print(df['Application_Name'].nunique())
```

```
6
```

```
print(df['Server_Name'].nunique())
```

```
4
```

```
print(df['User_ID'].nunique())
```

```
47641
```

NumPy Statistical Analysis

```
avg_response = np.mean(df['Response_Time_ms'])
print(avg_response)
```

```
2511.4177314285716
```

```
median_response = np.median(df['Response_Time_ms'])
print(median_response)
```

```
2511.0
```

```
max_response = np.max(df['Response_Time_ms'])  
print(max_response)
```

4999

```
min_response = np.min(df['Response_Time_ms'])  
print(min_response)
```

10

```
std_response = np.std(df['Response_Time_ms'])  
print(std_response)
```

1439.0807831580405

```
variance_response = np.var(df['Response_Time_ms'])  
print(variance_response)
```

2070953.500454759

```
p25 = np.percentile(df['Response_Time_ms'],25)  
print(p25)
```

1269.0

```
p50 = np.percentile(df['Response_Time_ms'],50)  
print(p50)
```

2511.0

```
p75 = np.percentile(df['Response_Time_ms'],75)  
print(p75)
```

3757.0

```
p95 = np.percentile(df['Response_Time_ms'],95)  
print(p95)
```

4753.0

```
p99 = np.percentile(df['Response_Time_ms'],99)  
print(p99)
```

4949.0

```
avg_cpu = np.mean(df['CPU_Usage_Percent'])  
print(avg_cpu)
```

50.5127648

```
avg_memory = np.mean(df['Memory_Usage_MB'])  
print(avg_memory)
```

8055.159108571429

```
avg_disk = np.mean(df['Disk_Usage_Percent'])  
print(avg_disk)
```

52.44923154285715

```
avg_request = np.mean(df['Request_Size_KB'])  
print(avg_request)
```

2500.3526365714292

```
print(df['Application_Name'].nunique())
```

6

```
requests_app = df.groupby('Application_Name').size()  
print(requests_app)
```

```
Application_Name
Banking      29070
CRM          28969
ECommerce    29272
ERP          29270
HRMS         29047
Inventory    29372
dtype: int64
```

```
errors = df[df['Log_Level'].isin(['ERROR','CRITICAL'])]
error_count = errors.groupby('Application_Name').size()
print(error_count)
```

```
Application_Name
Banking      5914
CRM          5755
ECommerce    5995
ERP          5928
HRMS         5788
Inventory    5919
dtype: int64
```

```
critical_errors = df[df['Log_Level']=='CRITICAL']
result = critical_errors.groupby('Application_Name').size()
print(result)
```

```
Application_Name
Banking      1480
CRM          1392
ECommerce    1448
ERP          1454
HRMS         1444
Inventory    1478
dtype: int64
```

```
slow_requests = df[df['Response_Time_ms'] > 3000]
result = slow_requests.groupby('Application_Name').size()
print(result)
```

```
Application_Name
Banking      11552
CRM          11688
ECommerce    11805
ERP          11707
HRMS         11701
Inventory    11854
dtype: int64
```

```
avg_response = df.groupby('Application_Name')['Response_Time_ms'].mean()
print(avg_response)
```

```
Application_Name
Banking      2497.968215
CRM          2516.043702
ECommerce    2514.695989
ERP          2512.176016
HRMS         2509.093641
Inventory    2518.442088
Name: Response_Time_ms, dtype: float64
```

```
peak_app = df['Application_Name'].value_counts().idxmax()
print(peak_app)
```

```
Inventory
```

```
least_app = df['Application_Name'].value_counts().idxmin()
print(least_app)
```

```
CRM
```

```
availability = ((df['Status_Code']==200).sum()/len(df))*100
print(availability)
```

```
12.485714285714286
```

```
success_rate = ((df['Status_Code']==200).mean())*100
print(success_rate)
```

```
12.485714285714286
```

```
failure_rate = ((df['Status_Code']>=400).mean())*100
print(failure_rate)
```

```
74.94800000000001
```

```
login_events = (df['Event_Type']=='Login').sum()
print(login_events)
```

```
24973
```

```
logout_events = (df['Event_Type']=='Logout').sum()
print(logout_events)
```

```
25098
```

```
search_events = (df['Event_Type']=='Search').sum()
print(search_events)
```

```
24752
```

```
api_calls = (df['Event_Type']=='API_Call').sum()
print(api_calls)
```

```
24823
```

Server Analysis

```
server_requests = df.groupby('Server_Name').size()
print(server_requests)
```

```
Server_Name
APP-SRV-01    44082
APP-SRV-02    43423
APP-SRV-03    43853
APP-SRV-04    43642
dtype: int64
```

```
cpu_server = df.groupby('Server_Name')['CPU_Usage_Percent'].mean()
print(cpu_server)
```

```
Server_Name
APP-SRV-01    50.477514
APP-SRV-02    50.850814
APP-SRV-03    50.262148
APP-SRV-04    50.463847
Name: CPU_Usage_Percent, dtype: float64
```

```
memory_server = df.groupby('Server_Name')['Memory_Usage_MB'].mean()
print(memory_server)
```

```
Server_Name
APP-SRV-01    8030.895059
APP-SRV-02    8048.122999
APP-SRV-03    8051.137961
APP-SRV-04    8090.709179
Name: Memory_Usage_MB, dtype: float64
```

```
disk_server = df.groupby('Server_Name')['Disk_Usage_Percent'].mean()
print(disk_server)
```

```
Server_Name
APP-SRV-01    52.482824
APP-SRV-02    52.491256
APP-SRV-03    52.473194
APP-SRV-04    52.349408
Name: Disk_Usage_Percent, dtype: float64
```

```
response_server = df.groupby('Server_Name')['Response_Time_ms'].mean()
print(response_server)
```

```
Server_Name
APP-SRV-01    2521.755388
APP-SRV-02    2517.657946
APP-SRV-03    2506.771464
APP-SRV-04    2499.435681
Name: Response_Time_ms, dtype: float64
```

```
errors = df[df['Log_Level'].isin(['ERROR', 'CRITICAL'])]
result = errors.groupby('Server_Name').size()
print(result)
```

```
Server_Name
APP-SRV-01    8944
APP-SRV-02    8716
APP-SRV-03    8781
APP-SRV-04    8858
dtype: int64
```

```
critical = df[df['Log_Level']=='CRITICAL']
result = critical.groupby('Server_Name').size()
print(result)
```

```
Server_Name
APP-SRV-01    2098
APP-SRV-02    2181
APP-SRV-03    2185
APP-SRV-04    2232
dtype: int64
```

```
ranking = df.groupby('Server_Name')['Response_Time_ms'].mean()
ranking = ranking.sort_values()
print(ranking)
```

```
Server_Name
APP-SRV-04    2499.435681
APP-SRV-03    2506.771464
APP-SRV-02    2517.657946
APP-SRV-01    2521.755388
Name: Response_Time_ms, dtype: float64
```

```
top_server = df.groupby('Server_Name')['Response_Time_ms'].mean().idxmin()
print(top_server)
```

```
APP-SRV-04
```

```
worst_server = df.groupby('Server_Name')['Response_Time_ms'].mean().idxmax()
print(worst_server)
```

```
APP-SRV-01
```

```
resource = df.groupby('Server_Name')[['CPU_Usage_Percent', 'Memory_Usage_MB', 'Disk_Usage_Percent']].mean()
print(resource)
```

Server_Name	CPU_Usage_Percent	Memory_Usage_MB	Disk_Usage_Percent
APP-SRV-01	50.477514	8030.895059	52.482824
APP-SRV-02	50.850814	8048.122999	52.491256
APP-SRV-03	50.262148	8051.137961	52.473194
APP-SRV-04	50.463847	8090.709179	52.349408

```
traffic = df['Server_Name'].value_counts()
print(traffic)
```

```
Server_Name
APP-SRV-01    44082
APP-SRV-03    43853
APP-SRV-04    43642
APP-SRV-02    43423
Name: count, dtype: int64
```

```
server_load = df.groupby('Server_Name').size()
print(server_load)
```

```

Server_Name
APP-SRV-01    44082
APP-SRV-02    43423
APP-SRV-03    43853
APP-SRV-04    43642
dtype: int64

```

```

availability = df.groupby('Server_Name').apply(lambda x:(x['Status_Code']==200).mean()*100)
print(availability)

```

```

Server_Name
APP-SRV-01    12.476748
APP-SRV-02    12.541741
APP-SRV-03    12.585228
APP-SRV-04    12.339031
dtype: float64
/tmp/ipykernel_4385/4275589953.py:1: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior
availability = df.groupby('Server_Name').apply(lambda x:(x['Status_Code']==200).mean()*100)

```

```

sla = df.groupby('Server_Name').apply(lambda x:(x['Response_Time_ms']<2000).mean()*100)
print(sla)

```

```

Server_Name
APP-SRV-01    39.426523
APP-SRV-02    39.327085
APP-SRV-03    39.844480
APP-SRV-04    40.021081
dtype: float64
/tmp/ipykernel_4385/834462466.py:1: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior i
sla = df.groupby('Server_Name').apply(lambda x:(x['Response_Time_ms']<2000).mean()*100)

```

User Analysis

```
print(df['User_ID'].nunique())
```

```
47641
```

```
active_users = (df['User_ID'].value_counts())
print(active_users)
```

```

User_ID
14994    13
27565    13
1342     13
14050    13
36043    13
..
13373     1
34815     1
40607     1
37971     1
43129     1
Name: count, Length: 47641, dtype: int64

```

```
top_users = (df['User_ID'].value_counts().head(10))
print(top_users)
```

```

User_ID
14994    13
27565    13
1342     13
14050    13
36043    13
29408    12
19006    12
43233    12
38626    12
13113    12
Name: count, dtype: int64

```

```
requests_user = df.groupby('User_ID').size()
print(requests_user)
```

```

User_ID
1000     4
1001     4

```



```
1002    2
1003    3
1004    4
..
49995   6
49996   2
49997   4
49998   3
49999   4
Length: 47641, dtype: int64
```

```
failed = df[df['Status_Code']>=400]
result = failed.groupby('User_ID').size()
print(result)
```

```
User_ID
1000    2
1001    3
1002    2
1003    2
1004    1
..
49995   6
49996   2
49997   3
49998   2
49999   3
Length: 45606, dtype: int64
```

```
session_analysis = df.groupby('Session_ID').size()
print(session_analysis)
```

```
Session_ID
SES100002    1
SES100005    1
SES100007    1
SES100009    1
SES100016    1
..
SES999971    2
SES999984    1
SES999987    1
SES999993    1
SES999997    1
Length: 159156, dtype: int64
```

```
avg_session = df.groupby('Session_ID').size().mean()
print(avg_session)
```

```
1.0995501269195003
```

```
login_frequency = df[df['Event_Type']=='Login'].groupby('User_ID').size()
print(login_frequency)
```

```
User_ID
1002    1
1003    1
1004    1
1006    1
1009    1
..
49989   2
49993   1
49995   1
49998   1
49999   2
Length: 19618, dtype: int64
```

```
user_trend = df.resample('D')['User_ID'].nunique()
print(user_trend)
```

```
Timestamp
2026-01-01    1417
2026-01-02    1425
2026-01-03    1423
2026-01-04    1420
2026-01-05    1415
...
2026-04-28    1417
2026-04-29    1422
```

```
2026-04-30    1418
2026-05-01    1418
2026-05-02     751
Freq: D, Name: User_ID, Length: 122, dtype: int64
```

```
engagement = (len(df)/df['User_ID'].nunique())
print(engagement)
```

```
3.673306605654793
```

Performance Analysis

```
response_trend = df.resample('H')['Response_Time_ms'].mean()
print(response_trend)
```

```
Timestamp
2026-01-01 00:00:00    2421.750000
2026-01-01 01:00:00    2648.616667
2026-01-01 02:00:00    2208.666667
2026-01-01 03:00:00    2622.666667
2026-01-01 04:00:00    2480.983333
...
2026-05-02 08:00:00    2636.283333
2026-05-02 09:00:00    2815.133333
2026-05-02 10:00:00    2306.250000
2026-05-02 11:00:00    2012.750000
2026-05-02 12:00:00    2354.625000
Freq: h, Name: Response_Time_ms, Length: 2917, dtype: float64
/tmp/ipykernel_4385/3309639328.py:1: FutureWarning: 'H' is deprecated and will be removed in a future version, please use 'h' in
response_trend = df.resample('H')['Response_Time_ms'].mean()
```

```
hourly_traffic = df.resample('H').size()
peak_hour = hourly_traffic.idxmax()
print(peak_hour)
```

```
2026-01-01 00:00:00
/tmp/ipykernel_4385/172088626.py:1: FutureWarning: 'H' is deprecated and will be removed in a future version, please use 'h' ins
hourly_traffic = df.resample('H').size()
```

```
daily_traffic = df.resample('D').size()
peak_day = daily_traffic.idxmax()
print(peak_day)
```

```
2026-01-01 00:00:00
```

```
avg_daily_requests = (df.resample('D').size().mean())
print(avg_daily_requests)
```

```
1434.4262295081967
```

```
avg_hourly_requests = (df.resample('H').size().mean())
print(avg_hourly_requests)
```

```
59.99314364072677
/tmp/ipykernel_4385/51450744.py:1: FutureWarning: 'H' is deprecated and will be removed in a future version, please use 'h' inst
avg_hourly_requests = (df.resample('H').size().mean())
```

```
slow_requests = len(df[df['Response_Time_ms'] > 3000])
print(slow_requests)
```

```
70307
```

```
sla_violations = len(df[df['Response_Time_ms'] > 2000])
print(sla_violations)
```

```
105572
```

```
print(df['Response_Time_ms'].describe())
```

```
count    175000.000000
mean      2511.417731
std       1439.084895
min        10.000000
```

```
25%      1269.000000
50%      2511.000000
75%      3757.000000
max       4999.000000
Name: Response_Time_ms, dtype: float64
```

```
errors = df[df['Log_Level'].isin(['ERROR','CRITICAL'])]
error_spikes = errors.resample('H').size()
print(error_spikes)
```

```
Timestamp
2026-01-01 00:00:00    15
2026-01-01 01:00:00    11
2026-01-01 02:00:00    17
2026-01-01 03:00:00    15
2026-01-01 04:00:00     9
..
2026-05-02 08:00:00    12
2026-05-02 09:00:00    13
2026-05-02 10:00:00    14
2026-05-02 11:00:00     5
2026-05-02 12:00:00     9
Freq: h, Length: 2917, dtype: int64
/tmp/ipykernel_4385/726630023.py:2: FutureWarning: 'H' is deprecated and will be removed in a future version, please use 'h' instead
  error_spikes = errors.resample('H').size()
```

```
cpu_spikes = df[df['CPU_Usage_Percent'] > 90]
print(cpu_spikes.shape[0])
```

```
17598
```

```
memory_spikes = df[df['Memory_Usage_MB'] > 12000]
print(memory_spikes.shape[0])
```

```
44020
```

```
disk_spikes = df[df['Disk_Usage_Percent'] > 90]
print(disk_spikes.shape[0])
```

```
18370
```

```
high_latency = df[df['Response_Time_ms'] > 4000]
print(high_latency.shape[0])
```

```
35183
```

```
df['Network_Load_KB'] = (df['Request_Size_KB']+df['Response_Size_KB'])
network_load = np.mean(df['Network_Load_KB'])
print(network_load)
```

```
7497.350456971429
```

```
bottlenecks = df[(df['Response_Time_ms'] > 3000)&(df['CPU_Usage_Percent'] > 80)]
print(bottlenecks.shape[0])
```

```
14201
```

Security Analysis

```
failed_login = len(df[(df['Event_Type']=='Login')&(df['Status_Code']>=400)])
print(failed_login)
```

```
18753
```

```
unauthorized = len(df[df['Status_Code']==401])
print(unauthorized)
```

```
22006
```

```
error401 = len(df[df['Status_Code']==401])
print(error401)
```

```
22006
```

```
error403 = len(df[df['Status_Code']==403])
print(error403)
```

21718

```
error404 = len(df[df['Status_Code']==404])
print(error404)
```

22119

```
error500 = len(df[df['Status_Code']==500])
print(error500)
```

21950

```
error503 = len(df[df['Status_Code']==503])
print(error503)
```

21581

```
critical_trend = df[df['Log_Level']=='CRITICAL'].resample('D').size()
print(critical_trend)
```

```
Timestamp
2026-01-01    77
2026-01-02    79
2026-01-03    78
2026-01-04    69
2026-01-05    71
..
2026-04-28    79
2026-04-29    67
2026-04-30    83
2026-05-01    94
2026-05-02    38
Freq: D, Length: 122, dtype: int64
```

```
failed_requests = df[df['Status_Code'] >= 400]
suspicious_users = (failed_requests.groupby('User_ID').size().sort_values(ascending=False).head(20))
print(suspicious_users)
```

```
User_ID
14994    12
34863    11
36043    11
25777    11
27565    11
47219    11
16863    10
19760    10
36618    10
6422     10
27104    10
3775     10
1342     10
43233    10
14050    10
18555    10
38310    10
30536    10
38626    10
10410    10
dtype: int64
```

```
error_sources = df[df['Status_Code'] >= 400]
result = error_sources.groupby('Module').size()
print(result)
```

```
Module
API          21785
Authentication  21896
Database      21961
Orders        21793
Payment       21788
Reporting     21936
dtype: int64
```

```
attack_indicators = (failed_requests.groupby('Host_IP').size())
attack_indicators = attack_indicators[attack_indicators > 100]
print(attack_indicators)
```

```
Series([], dtype: int64)
```

```
top_error_modules = (error_sources.groupby('Module').size().sort_values(ascending=False).head(10))
print(top_error_modules)
```

```
Module
Database      21961
Reporting      21936
Authentication 21896
Orders         21793
Payment        21788
API            21785
dtype: int64
```

```
db_failures = df[(df['Module']=='Database')&(df['Status_Code']>=400)]
print(db_failures.shape[0])
```

```
21961
```

```
api_failures = df[(df['Module']=='API')&(df['Status_Code']>=400)]
print(api_failures.shape[0])
```

```
21785
```

```
security_alerts = len(df[df['Log_Level']=='CRITICAL'])
print(security_alerts)
```

```
8696
```